

ネットワーク及び演習

第 2 回 計算機ネットワークの基礎、Python のデータ型

佐野 彰

本日の到達目標

計算機ネットワークの基礎

- 通信にプロトコル（決まり）が要る理由を、「情報とは何か」から説明できる
- プロトコル階層の各層を挙げ、それぞれの役割を説明できる
- 自分の PC の IP アドレスを調べ、プライベートかグローバルか判定できる
- ドメイン名から IP アドレスを調べる仕組み（DNS）を説明できる

Python のデータ型

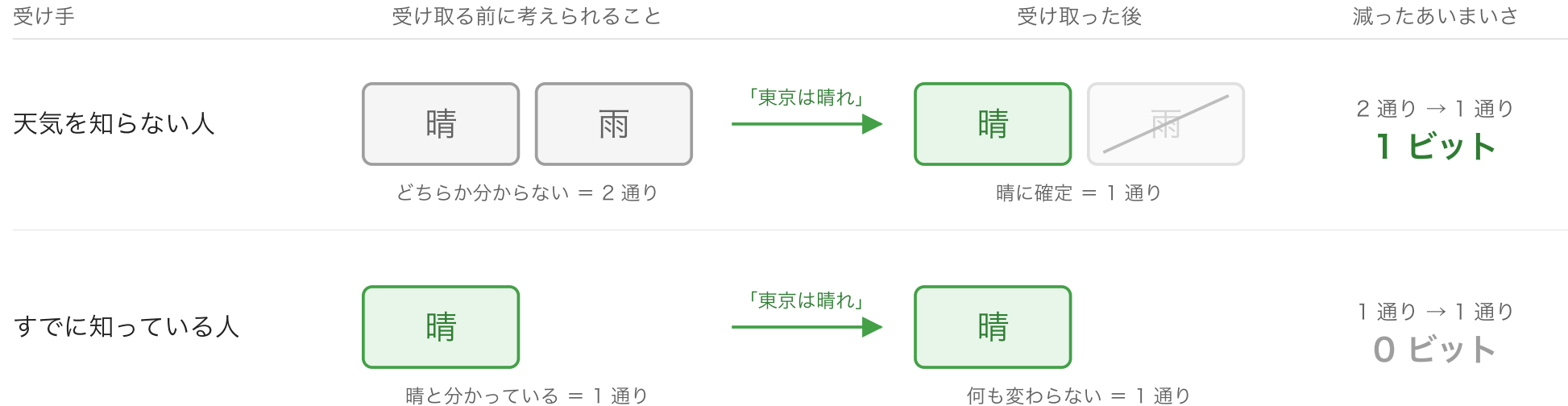
- 値には型があり、`input()` が返すのは常に `str` であることを説明できる
- 文字列とリストを、同じ「並び」として扱える
- スライス `s[開始:終了:刻み]` を使って ex2 の 4 問を解ける

講義パートのあとに小テスト（前回の範囲・16:45～17:00 の 15 分、1-608 でのみ受験可）。5 講時は実習。

1. 計算機ネットワークの基礎

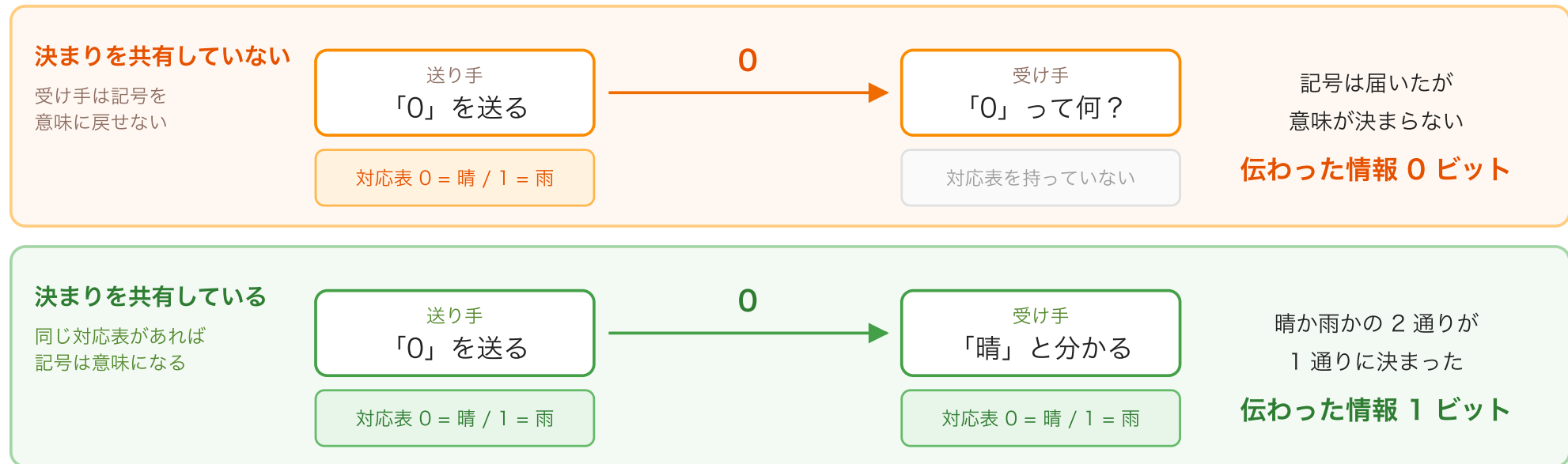
情報 → プロトコル → 階層 → アドレス → 名前

情報とは — 受け手のあいまいさが減った量



- 情報には必ず送り手と受け手がいて、情報量は受け手の側で決まる
- 2 通りの可能性が 1 通りに決まったら 1 ビット。あいまいさが減らなければ 0 ビット
- 通信とは、受け手のあいまいさを減らすために信号を送ること

伝えるには、送り手と受け手が決まりを共有している必要がある



- 同じ意味でも表現はさまざま (Merci・Gracias・ありがとう)。受け手が解釈できて初めて情報になる
- 送り手と受け手が共有するこの決まりを **プロトコル (protocol)** と呼ぶ

プロトコルは誰が決めているのか

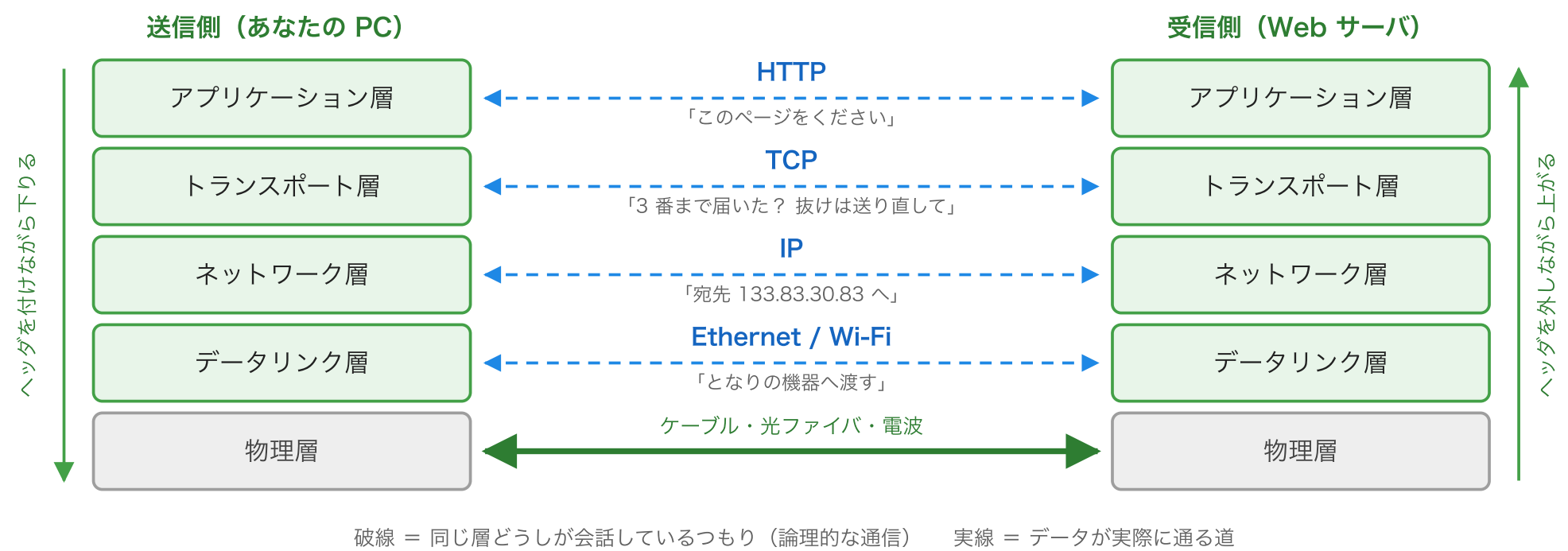
- プロトコル = あいまいさを無くするための手続きと約束事
 - 記号の意味だけでなく、やり取りの順番や、失敗したときの振る舞いまで決める
- 身近な例
 - 算術の四則演算（記号の意味、演算の優先順位）
 - 京都議定書（Kyoto **Protocol**, 1997）、パリ協定 — 国際的な取り決めも protocol
- インターネットの通信プロトコル → **TCP/IP**
 - **T**ransmission **C**ontrol **P**rotocol / **I**nternet **P**rotocol
 - **IETF**（Internet Engineering Task Force）という非政府の組織が、**RFC**（Request For Comments：意見求む！）という文書で公開している
 - 誰でも読める。たとえば TCP は RFC 9293、HTTP/1.1 は RFC 9112

役割ごとに層に分ける — プロトコル階層

層	役割	例
アプリケーション層	何を伝えるか	HTTP, SMTP, DNS
トランスポート層	相手のプログラム（ポート）まで届ける。TCP は順序と到達も保証する	TCP , UDP
ネットワーク層	相手のネットワークまで届ける	IP, ICMP
データリンク層	隣の機器へ渡す	Ethernet, Wi-Fi
物理層	信号にして送る	ケーブル、光ファイバ、電波

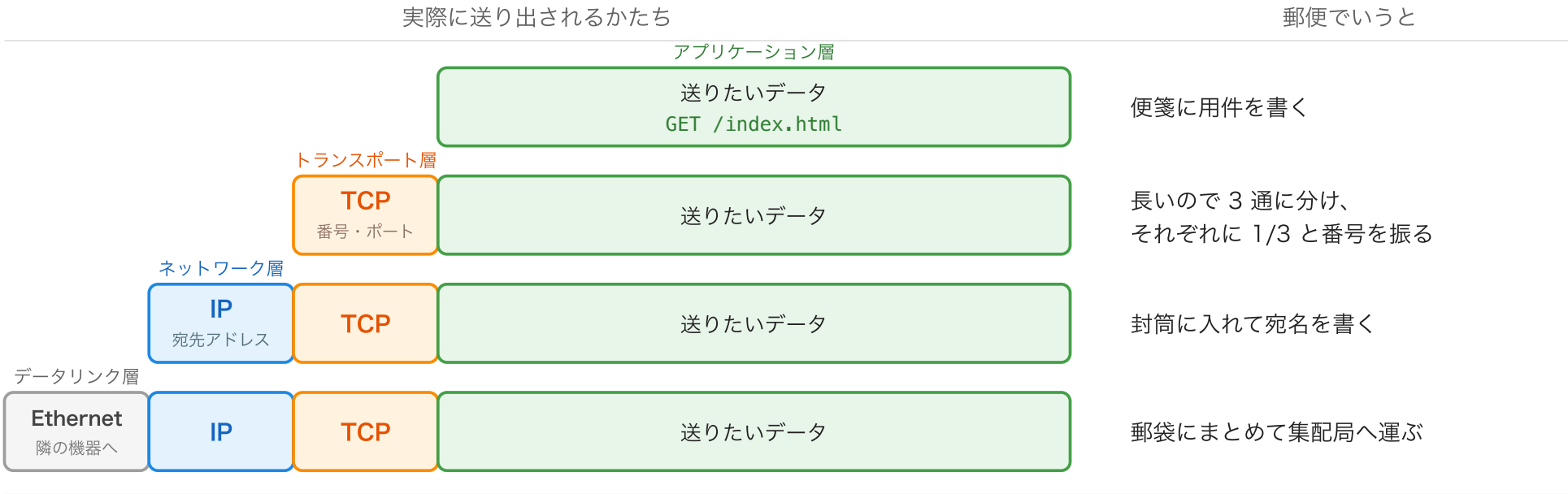
- 上の層ほど人間に近く、下の層ほど機械に近い
- 層に分けてあると、1 つの層だけを取り替えられる。Wi-Fi でも有線でも、同じ HTTP がそのまま動く
- この科目で書くのは**アプリケーション層**。TCP と IP の処理は OS が行い、Python からは `socket` で使う

同じ層どうしが会話している



- 各層は、相手の同じ層とやり取りしているものとして作れる。下の層の運び方は気にしない
- 実際にデータが流れるのは物理層だけ。上の層どうしのやり取りは、下の層の働きの上に成り立つ

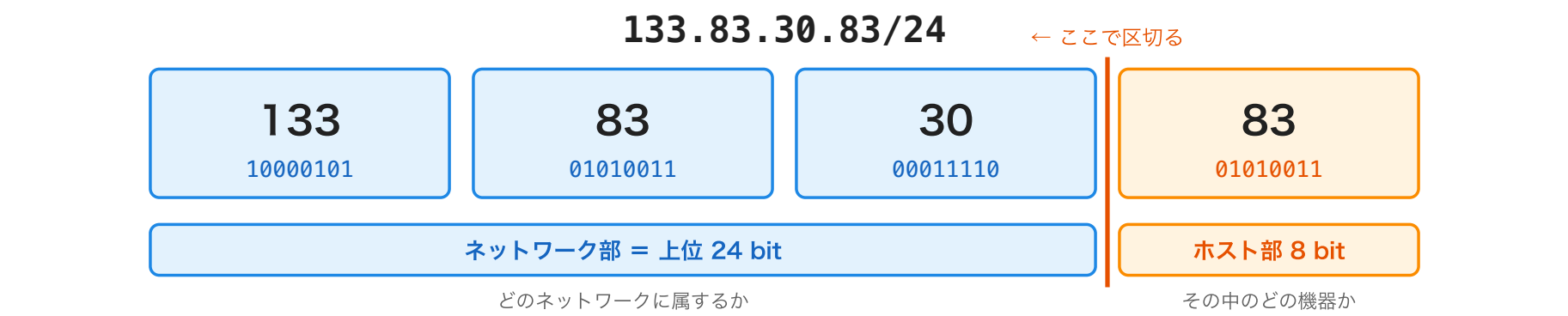
下りながらヘッダを付ける — カプセル化



受信側では、外側に付いたヘッダから順に外していく。集配局が便箋を読まないのと同じで、どの層も自分が付けたヘッダしか見ない。だから中身が何であっても運べる。

- 送信側では、層を下りるたびにその層のヘッダがデータの前に付く（カプセル化）
- 受信側は外側のヘッダから順に外す。どの層も自分のヘッダしか読まないで、中身が何でも運べる

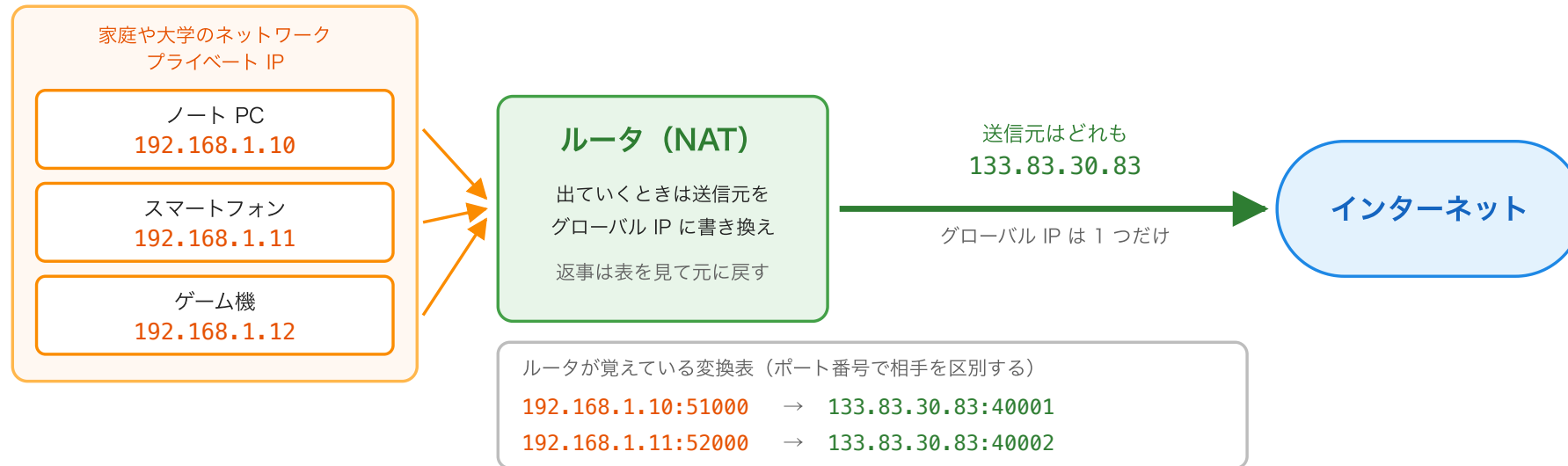
IP アドレス — ネットワーク上の住所



ネットワーク部が同じ機器どうしは直接やりとりできる。違えばルータが中継する。
/24 はサブネットマスク 255.255.255.0 と同じ意味。IPv6 は 128 bit で 2001:2f8:1001:a024:133::7 のように書く。

- IPv4 は 32 bit (2^{32} = 約 43 億個)、IPv6 は 128 bit。IPv4 のアドレスは枯渇し、IPv6 への移行が進む
- 住所にたとえると、ネットワーク部が町名、ホスト部が番地にあたる。どこで区切るかを /24 のように書く
- 割り当てを管理するのは、世界では ICANN、日本では JPNIC。自分で勝手に決めて使ってよい番号ではない

1 つのグローバル IP を分け合う — NAT



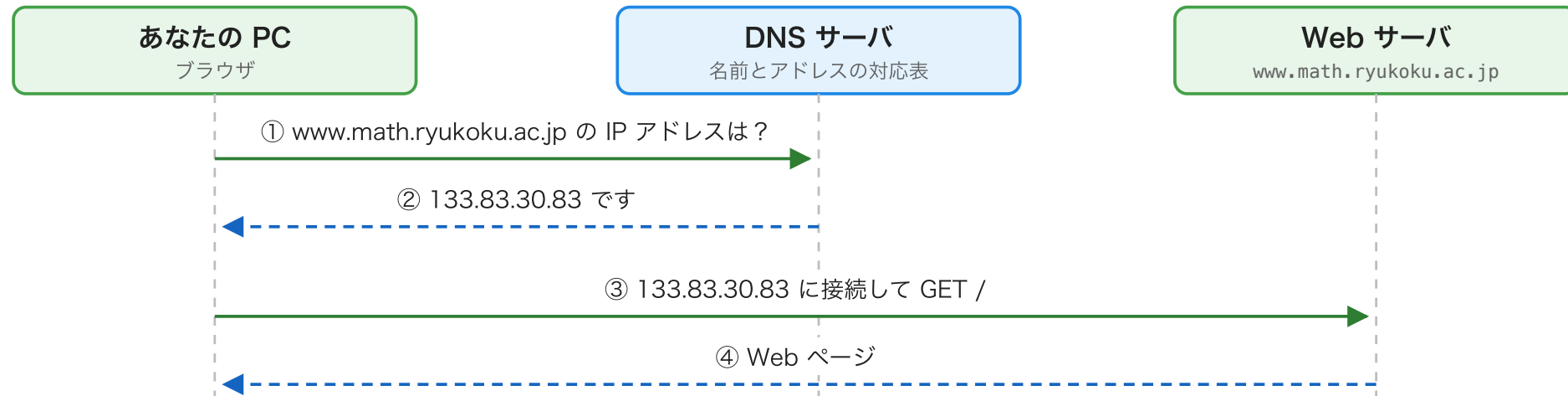
- グローバル IP はインターネット上で一意、プライベート IP はそのネットワークの中だけで一意
 - 192.168.x.x / 10.x.x.x / 172.16~31.x.x は誰が使ってもよい番号
- IPv4 が 43 億個で足りているのは NAT のおかげ。ただし、NAT の内側の PC は、そのままでは外から接続を受けられない

自分の PC のアドレスを調べる (5 講時)

OS	コマンド
Windows	<code>ipconfig</code>
macOS	<code>ipconfig getifaddr en0</code> (一覧は <code>ifconfig</code>)
Linux / WSL	<code>ip address</code>

- 表示された値はプライベートか、グローバルか？ (`192.168.` `10.` `172.16~31.` ならプライベート)
- グローバル側のアドレスは `確認くん` で分かる。手元の値と違っていることを確かめる
- 値が違っていれば、途中のルータが NAT を行っている

名前で呼べるようにする — DNS



人はドメイン名で指定し、機械は IP アドレスで通信する。その橋渡しをするのが DNS。

- 覚えにくい IP アドレスに、人間向けの名前を付けたものがドメイン名
- `nslookup www.math.ryukoku.ac.jp` (`host` コマンドでも可) で対応を調べられる

2. Python のデータ型

書き方は paiza、動きは教材ノートブックで確かめる

値には型がある — 動的型付け

```
n = 5          # int    整数
x = 1.5        # float  小数
s = 'Ryukoku'  # str    文字列
b = True       # bool   真偽値
a = [1, 2, 3]  # list   並び

print(type(n)) # <class 'int'>
```

- 宣言は要らない。代入した値で型が決まる
- 型は値の側にある。たとえば、変数は入れ物ではなく値に付けた名札

```
n = input()    # '5' ← str のまま
print(n * 2)    # 55   文字列が 2 回

n = int(input()) # 5    ← 整数にする
print(n * 2)    # 10
```

`input()` が返すのは常に `str`。数として使うなら `int()` や `float()` で変換する。ここを間違えると、ex2 のテストには通らない。

文字列とリスト — どちらも「並び」

```
s = 'Ryukoku'
a = ['Ryu', 'koku']

len(s), len(a)    # 7 2
s[0], a[0]        # 'R' 'Ryu'
'yu' in s         # True
s + '!'           # 'Ryukoku!'
'-' * 3           # '---'
```

- 番号でのアクセス・`len()`・`in`・`+`・`*` は、どちらでも同じように使える
- 両方に同じ書き方が使えるので、覚えることが少なくて済む

```
a = [10, 20, 30]
a.append(40)      # [10, 20, 30, 40]
a[1] = 99         # 書き換えられる

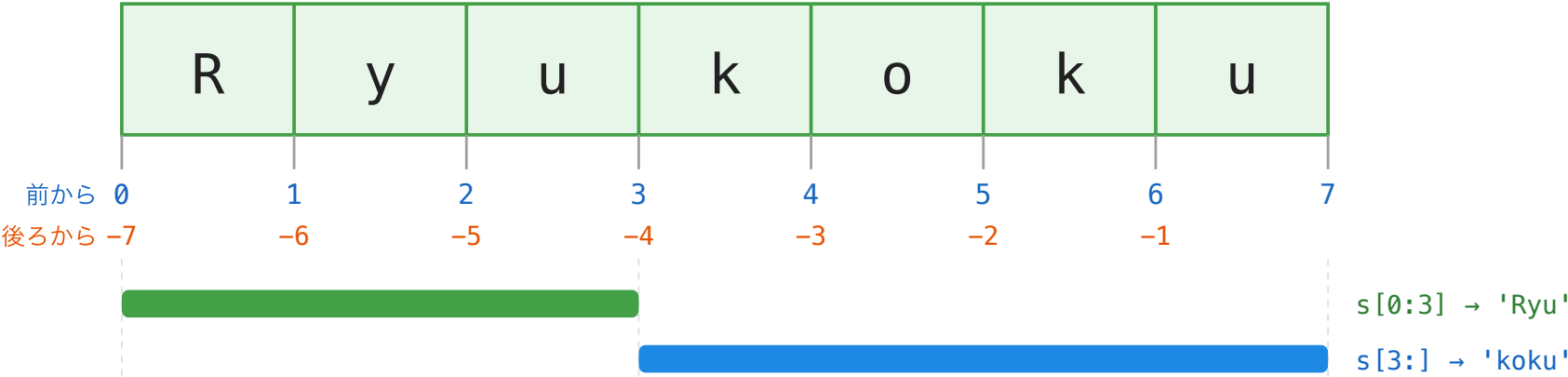
s = 'Ryukoku'
s[0] = 'r'        # TypeError
```

- リストは書き換えられる。文字列は書き換えられない
- 文字列を変えたいときは、新しい文字列を作る

番号は「境目」に付く — スライス

```
s = 'Ryukoku'
```

番号は文字そのものではなく、文字と文字の「境目」に付いている



s[開始:終了] は「開始の境目から終了の境目まで」。だから終了の番号の文字は入らない。

s[0:3]	# 'Ryu'	s[:3]	# 'Ryu' (省略 = 端まで)	s[3:]	# 'koku'
s[-1]	# 'u'	s[::-2]	# 'Ruou' (2 文字おき)	s[::-1]	# 'ukokuyR' (逆順)

繰り返し — for と range

```
for i in range(5):    # 0 1 2 3 4
    print(i)

for c in 'Ryu':       # R y u
    print(c)

for x in [10, 20]:    # 10 20
    print(x)
```

- `range(n)` は `n` を含まない (C の `i < n` と同じ)
- `for` は並びから要素を 1 つずつ取り出す構文。C の添字ループとは考え方が違う

```
n = int(input())
s = input()
for i in range(n):
    print(s)
```

- ex2 の `nstring.py` はこれで書ける

```
while True:           # 第 3 回以降で使う
    data = sock.recv(1024)
    if not data:
        break
```

- 回数が決まらないときは `while` と `break`

今日やること・次回

5 講時（1-608 で実習）

1. 自分の PC の IP アドレスを調べ、プライベートかグローバルか確かめる
2. paiza ラーニング「新・Python入門編」5～8（4 レッスン）の認定証を、まとめて 1 回で aiJudge へ提出
3. 教材ノートブック `python_string-list.py` でスライスを実際に動かす
4. aiJudge の ex2（4 問: `nstring.py` ・ `rstlist.py` ・ `rstring.py` ・ `stepstring.py`）すべてで満点を取る。
締切は 10/02（金）0:00

aiJudge は提出のたびに自動採点する。満点になるまで何度でも提出してよい。

次回（10/02）：echo クライアント・サーバ、Python の関数

質問は Teams 「★ 質問用チャンネル」 か、教室の TA へ。